

CS 5330 Final Project: Transfer Learning versus Random Initialization

Isolating feature quality from trainable capacity in scene classification

Team

- **Shriman Raghav Srinivasan** — srinivasan.shrim@northeastern.edu
- **Group members:** none. This is an individual project; all code, experiments, report, and slides are my own work.
- Khoury College of Computer Sciences, Northeastern University
- CS 5330 Pattern Recognition and Computer Vision, Summer 2026, Prof. Bruce Maxwell

Project Description

Transfer learning reliably beats training from scratch, but the standard comparison moves two variables at once: the **quality of the pretrained features** and the **number of parameters left trainable**. When a pretrained network with frozen early layers outperforms a network trained from scratch, both explanations predict exactly the same headline number, so the experiment everyone runs cannot say which one is responsible.

This project adds the missing control. On six-class natural scene classification with ResNet-18, I run three conditions that differ only in how the early block θ_f is treated:

Condition	Early block θ_f		Trainable	Purpose
Pretrained-Frozen (PT-F)	ImageNet	weights, frozen	layer4 + fc , 8,396,806	the transfer-learning arm
Random-Frozen (R-F)	random	weights, frozen	layer4 + fc , 8,396,806	capacity-matched control
Random-Full (R-FL)	random	weights, trained	everything, 11,179,590	train-from-scratch arm

R-F is the point of the design. It has exactly the same trainable capacity as PT-F and differs only in whether the frozen weights carry ImageNet knowledge, so **PT-F minus R-F is feature quality with capacity held fixed**, and **R-F minus R-FL is capacity with initialization held random**. Two clean comparisons instead of one confounded one.

Research objectives

1. Quantify feature quality with capacity fixed — PT-F against the capacity-matched control.
2. Quantify the contribution of capacity — the frozen random control against a fully trainable network.
3. Locate where fine-tuning stops paying — a freeze-depth sweep from linear probe to full fine-tuning.
4. Measure the first layer's reach — replace `conv1` with a fixed Gabor bank, then unfreeze upward.

A critical implementation detail: `requires_grad = False` does *not* freeze BatchNorm. Running statistics keep updating on every forward pass, so a block believed to be frozen quietly drifts. Both frozen conditions hold BatchNorm in `eval()` mode; Random-Full updates normally, by design.

Protocol. Stratified 90/10 train/validation split (12,631 / 1,403 images), 3,000 held-out test images. SGD at lr 1e-3, momentum 0.9, weight decay 1e-4, batch size 32. Three conditions $\times$ three seeds (42, 100, 2026), early stopping on validation loss with patience 5. Nine runs total, evaluated once on the test split.

Key Results

Condition	Mean test acc.	Std. dev.	Epochs	Wall clock
Pretrained-Frozen	93.27%	0.23%	8.0 ± 0.8	103 s
Random-Full	84.69%	1.87%	15.0 ± 2.2	426 s
Random-Frozen	66.54%	4.04%	8.7 ± 2.1	113 s

- **Pretrained features are worth +26.7 points** at matched capacity (PT-F vs. R-F). Identical trainable parameters, data, and recipe; the only difference is where the frozen weights came from.
- **Capacity on its own is worth +18.1 points** (R-F vs. R-FL), and still finishes 8.6 points short of PT-F at roughly four times the wall-clock cost.
- Both factors matter. The representation matters considerably more.

Freeze-depth ablation. A bare linear probe on frozen pretrained features already reaches **91.3% from 3,078 trainable parameters**. Unfreezing **layer4** adds 1.6 points to 92.9%; **layer3**, **layer2**, and full fine-tuning are all flat at 92.8%. The elbow sits exactly at **layer4**, so most of pretraining’s value needs almost no fine-tuning. (*40% training subset, seed 42.*)

Gabor first-layer substitution. Swapping **conv1** for a fixed bank of 8 orientations $\times$ 4 wavelengths $\times$ 2 phases costs about 10 points at read-out (81.1% vs. 91.6%). Unfreezing **layer1** alone returns 5 of them; **layer2** through **layer4** add nothing. **Roughly 7 points never return**, so a hand-designed filter bank plus unrestricted downstream retraining cannot reconstruct what a learned first layer provides. (*Full data, seed 42.*)

Error structure. Glacier $\leftrightarrow$ Mountain and Buildings $\leftrightarrow$ Street dominate the confusions in every condition, structurally identical across conditions and merely larger under random initialization. These come from genuine visual overlap in the data, not from the training setup.

Deliverables and URLs

Item	Link
Demo video	Google Drive — also in-repo at <code>presentation/demo_video.mp4</code> (2:05, 1080p, silent with on-screen captions)
Final presentation	Google Drive — also in-repo at <code>presentation/Final_Presentation-PRCV.pdf</code> (12 slides)
Dataset (Kaggle)	Intel Image Classification
Dataset (direct zip mirror)	HuggingFace mirror
Report	<code>report/report.pdf</code> (7 pp, IEEE conference format), source <code>report/report.tex</code>
Checkpoints and metrics	<code>results/best_model_{condition}_seed{seed}.pth</code> , <code>results/summary_metrics.json</code> , <code>results/test_evaluation_metrics.json</code>
Figures	<code>results/</code> — <code>loss_curves.png</code> , <code>accuracy_curves.png</code> , <code>confusion_matrices.png</code> , <code>learning_curves.png</code> , <code>feature_pca.png</code> , <code>ablation_curve.png</code> , <code>gabor_curve.png</code>

Both video URLs are also embedded in the compiled report under **Supplementary Material**.

Dataset summary. Intel Image Classification: 150×150 natural scene images across six classes (buildings, forest, glacier, mountain, sea, street), 14,034 training and 3,000 test images. An unlabeled `seg_pred/` split ships with the dataset but is not used in this study.

Code Directory Structure

```
final_project/
data/          # Dataset files (seg_train/, seg_test/, seg_pred/)
src/
  __init__.py
  dataset.py   # Preprocessing transforms and stratified validation splits
  model.py     # ResNet-18 setups, parameter freezing, BN eval enforcement
  train.py     # Training loop with early stopping logic
  evaluate.py  # Evaluation on test split, precision/recall/F1 calculations
  run_experiments.py # Grid runner script (runs 9 jobs)
  visualize.py # Plots training curves and confusion matrix heatmaps
  generate_latex_results.py # Outputs results_summary.tex macros from JSON summaries
  ablation_freeze_depth.py # Extension: freeze-boundary sweep (fc -> layer4 -> ... -> full)
  gabor_ablation.py # Extension: fixed Gabor conv1 vs. learned conv1, unfreeze sweep
  make_extra_figures.py # Learning curves, feature PCA, and both ablation curve figures
results/      # Checkpoints, metric JSON files, and image plots
  ablation/   # JSON results and run logs for both extension experiments
report/       # LaTeX source, bibliography, and compiled report
presentation/ # Slide deck, assets, demo video, and their generators
proposal/     # Original project proposal
main.py       # Entry point: --run / --eval / --compile
verify_pipeline.py # Mock pipeline testing utility
```

Every number in the report is generated from the metrics JSON by `generate_latex_results.py`, which emits LaTeX macros, so the report cannot silently disagree with the runs.

Setup and Running Instructions

1. Requirements

Training and evaluation need PyTorch, torchvision, numpy, and matplotlib:

```
| pip install torch torchvision numpy matplotlib
```

Rebuilding the slide deck additionally needs `python-pptx` and `Pillow`, plus LibreOffice for the PPTX-to-PDF step. Compiling the report needs a TeX distribution providing `pdflatex` and `bibtex`. Building the demo video needs `ffmpeg` with `libx264`.

```
| pip install python-pptx Pillow
```

2. Dataset Acquisition

Download the dataset from one of the links above and place the extracted `seg_train/`, `seg_test/`, and `seg_pred/` folders under `data/`. There is no automatic download step.

3. Verification Check

Run the quick integration test (takes ~5 seconds) to verify model partitioning, batch norm handling, and check that your environment runs smoothly:

```
| python3 verify_pipeline.py
```

4. Running the Full Experiment Grid

To run the full 9 training experiments (3 conditions $\times$ 3 seeds) with early stopping, execute:

```
python3 main.py --run
```

All checkpoints and training curve data will be saved under `results/`.

5. Running Evaluation & Generating Plots

To run evaluation on the held-out test split and create the confusion matrices and learning curves, run:

```
python3 main.py --eval
```

This updates the summary files `results/summary_metrics.json` and `results/test_evaluation_metrics.json`, and outputs the plots `loss_curves.png`, `accuracy_curves.png`, and `confusion_matrices.png`.

6. Running the Extension Experiments (Ablations)

These two experiments are not part of the `main.py` grid and are run as standalone modules from the project root. Results land in `results/ablation/`.

Freeze-depth sweep on pretrained ResNet-18 (fc linear probe $\rightarrow$ `layer4` $\rightarrow$ `layer3` $\rightarrow$ `layer2` $\rightarrow$ full fine-tune). Defaults to a 40% stratified training subset at a single seed to stay tractable on CPU; use `-frac 1.0 -seeds 42 100 2026` on a GPU for full rigor:

```
python3 -m src.ablation_freeze_depth
```

Fixed Gabor first-layer substitution versus the learned first layer, swept over unfreeze depth (full data, single seed):

```
python3 -m src.gabor_ablation --seeds 42
```

7. Generating the Supplementary Figures

Learning curves and the penultimate-feature PCA are built from existing checkpoints (no retraining); the two ablation curves are built from the JSON files written in step 6:

```
python3 -m src.make_extra_figures          # learning_curves.png, feature_pca.png
python3 -m src.make_extra_figures --ablation # ablation_curve.png
python3 -m src.make_extra_figures --gabor   # gabor_curve.png
```

8. Compiling the Report & Slides

To generate the automated results summary macro file and build both deliverables:

```
python3 main.py --compile
```

This writes `report/results_summary.tex` from the metric JSON files, runs `pdflatex/bibtex` to produce `report/report.pdf`, then runs `presentation/make_pptx.py` to build `presentation/presentation.pptx` and converts it to `presentation/presentation.pdf` with headless LibreOffice. The PDF conversion is skipped with a warning if `soffice` is not on your PATH.

Slide figures are regenerated separately when the underlying results change:

```
python3 presentation/generate_assets.py
```

9. Building the Demo Video

Renders `presentation/demo_video.mp4` (1920×1080, 30 fps, ~2 minutes, silent with burned-in captions). Requires `ffmpeg` with `libx264`, plus the trained checkpoints and all figures from steps 5 and 7:

```
| python3 presentation/make_demo_video.py
```

The first segment loads the Pretrained-Frozen and Random-Frozen checkpoints (seed 42) and classifies the same held-out `seg_test` images side by side, showing the full softmax distribution for each. Because both models train an identical 8,396,806 parameters, every disagreement is attributable to initialization alone. The image sample is a fixed-seed random draw (`-seed 42`), not hand-picked. The second segment is a captioned pass over the result figures. Useful flags: `-n-images`, `-fps`, `-seed`, `-dry-run`.

Limitations

Stated in full in Section VI of the report, and summarized here:

- **One dataset, one architecture.** The 26.7-point figure is not claimed to generalize across domains or to deeper networks.
- **Three seeds, no significance test.** The gaps between conditions far exceed the within-condition spread, but no formal test is run. Both ablations are single-seed; read the shape of those curves, not their exact values.
- **No data augmentation.** The preprocessing pipeline is a single deterministic resize and centre crop shared by all three splits. This keeps the conditions on an identical input distribution but depresses absolute accuracy relative to tuned baselines, and disadvantages Random-Full in particular.
- **The Gabor bank is untuned.** Its parameters were fixed a priori rather than searched, so the residual 7-point gap is an upper bound.
- **The control is not a method.** Random-Frozen exists to hold capacity fixed. Its 66.5% is not a recommended configuration.

Acknowledgments

Prof. Bruce Maxwell (CS 5330, Khoury College) — his question about how far the influence of the first layer reaches up the network became the Gabor experiment. The Intel Image Classification dataset is distributed publicly via Kaggle. The ResNet-18 implementation and ImageNet-pretrained weights come from PyTorch and torchvision.